Intelligenza artificiale

Mattia Nicolis Matteo Drago

A.A. 2025-26

Indice

Introduzione all'intelligenza artificiale **5**
Tipologie di intelligenza artificiale 6
Agenti autonomi 6
Data analysis 6
Machine Learning 6
Time series analysis 6
Agenti intelligenti 7
Markov Decision Process (MDP) 7
Generative AI 7

Agenti intelligenti **9**
Misure di prestazione 10
Razionalità 10
Specificare un ambiente o (PEAS) 11

Risolvere i problemi con la ricerca **13**
Agenti risolutori di problemi 13
Problemi di ricerca e soluzini 13
Algoritmi di ricerca 13
Tree search 13
Misurare le prestazioni nella risoluzione di problemi 14
Cammini rindondanti 15
Strategie di ricerca non informata 15
Ricerca in ampiezza 15
Ricerca a costo minimo (algoritmo Dijkstra) 16
Ricerca in profondità 16
Ricerca a profondità limitata 17
Strategie di ricerca informata 18
Ricerca best-first greedy o "golosa" 19
Ricerca A* 19
Funzioni euristiche 20
Effetto dell'accuratezza dell'euristica sulle prestazioni 20
Generare euristiche da problemi rilassati 20

Introduzione all'intelligenza artificiale

Il compito dell'**intelligenza artificiale** (IA) non è solo quello di *comprendere*, ma anche di *costruire* entità intelligenti: macchine in grado di calcolare come agire in modo efficace e sicuro in un'ampia varietà di situazioni nuove.

In passato, gli studiosi hanno proposto diverse versioni di IA: alcuni hanno definito l'intelligenza in termini di fedeltà alla **prestazione umana**, mentre altri preferiscono una definizione formale di intelligenza come **razionalità**.

I metodi utilizzati sono necessariamente diversi: l'approccio che persegue un'intelligenza simile a quella umana deve essere in parte una scienza empirica correlata alla psicologia, richiedendo osservazioni e ipotesi riguardo al comportamento umano e ai processi di pensiero.

Un approccio razionalista, invece, sfrutta una combinazione di matematica e ingegneria e si collega alla statistica, alla teoria del controllo e all'economia.

Test di Turing

Il **test di Turing**, proposto da Alan Turing nel 1950, mira a comprendere se una macchina è in grado di pensare.

Un computer supera il test se un esaminatore umano, dopo aver posto alcune domande in forma scritta, non è in grado di distinguere se le risposte provengano da una persona o da una macchina.

Per superare il test, il computer deve possedere le seguenti capacità:

- **interpretazione del linguaggio naturale**, per comunicare con successo nel linguaggio umano
- **rappresentazione della conoscenza**, per memorizzare ciò che conosce o apprende
- **ragionamento automatico**, per rispondere alle domande e trarre nuove conclusioni
- **apprendimento automatico** (*machine learning*), per adattarsi a nuove circostanze, individuare ed estrapolare schemi

Esiste, inoltre, un cosiddetto **test di Turing totale**, che richiede l'interazione con oggetti e persone nel mondo reale.

Per superare il test di Turing totale, un robot deve anche possedere:

- **visione artificiale** e riconosciemtno vocale, per percepire il mondo

- **robotica**, per manipolare gli oggetti e muoversi fisicamente

Tipologie di intelligenza artificiale

Agenti autonomi

Sono sistemi che percepiscono l'ambiente e agiscono in modo autonomo per raggiungere obiettivi specifici.

Data analysis

Consiste nell'utilizzo di algoritmi per analizzare grandi quantità di dati ed estrarre informazioni utili e correlazioni complesse.

Machine Learning

È lo sviluppo di algoritmi che permettono ai modelli di apprendere dai dati di esempio e migliorare le proprie prestazioni nel tempo, senza essere esplicitamente programmati.

Un esempio è il riconoscimento di immagini.

L'apprendimento automatico si divide in tre categorie principali:

- **unsupervised learning**: l modello viene addestrato su dati non etichettati, con l'obiettivo di scoprire strutture nascoste o pattern nei dati senza risposte predefinite.
- **supervised learning**: l modello viene addestrato su dati etichettati, dove ogni input è associato a una risposta corretta. L'obiettivo è imparare a mappare correttamente gli input alle risposte
- **reinforced learning**: il modello apprende attraverso interazioni con l'ambiente, ricevendo ricompense o penalità in base alle azioni compiute. L'obiettivo è massimizzare la ricompensa totale nel tempo

Time series analysis

L'analisi delle serie temporali è un'area dell'apprendimento automatico che si concentra sull'analisi di dati raccolti nel tempo.

Le serie temporali sono sequenze di dati misurati a intervalli regolari, come la temperatura giornaliera, i prezzi delle azioni o i dati di vendita mensili.

L'obiettivo è identificare pattern, tendenze e stagionalità per effettuare previsioni future.

Gli approcci comuni includono:

- **riconoscimento di anomalie e cause**: identificazione di eventi che si discostano dal comportamento normale, potenzialmente indicativi di errori o problemi.
- **generative transformers**: modelli in grado di predire il prossimo elemento in una sequenza di dati (ad esempio la parola successiva in una frase o il pixel successivo in

un'immagine), utilizzando il concetto di **attenzione** per pesare l'importanza delle diverse parti della sequenza di input.

Agenti intelligenti

Un agente intelligente è un sistema che percepisce l'ambiente circostante attraverso sensori e agisce su di esso per raggiungere un obiettivo specifico.

Gli elementi chiave di un agente intelligente sono:

- **performance measure**: misura il successo dell'agente nel raggiungere i propri obiettivi
- **rationality**: l'agente deve agire in modo da massimizzare la performance attesa

Markov Decision Process (MDP)

Un MDP è un modello matematico utilizzato per rappresentare problemi di decisione sequenziali. I suoi elementi principali sono:

- **state**: rappresenta l'ambiente in un determinato momento
- **actions**: nsieme delle azioni che l'agente può intraprendere
- **transition model**: descrive l'effetto delle azioni sull'ambiente (può essere parzialmente incognito)
$$T : (state, action) \rightarrow next_state$$
- **reward**: vvalore immediato associato all'esecuzione di un'azione
$$R : (state, action, next_state) \rightarrow real_number$$
- **Policy**: strategia con cui l'agente decide quale azione intraprendere in ogni stato per massimizzare la ricompensa totale attesa nel tempo
$$\pi : (state) \rightarrow action$$

Generative AI

L'intelligenza artificiale generativa comprende una classe di modelli in grado di creare nuovi contenuti — testo, immagini, musica o video — a partire da dati di addestramento.

Questi modelli possiedono miliardi di parametri e sono pre-addestrati su enormi quantità di dati.

In sostanza, "predicono il futuro" basandosi sui dati su cui sono stati **addestrati** e su un **prompt** (input dell'utente).

Agenti intelligenti

Un **agente** è qualsiasi entità che possa essere vista come un sistema che percepisce il proprio ambiente attraverso dei **sensori** e agisce su di esso mediante **attuatori**.

Un agente umano possiede come **sensori** gli occhi e altri organi, e può utilizzare come **attuatori** mani, gambe, corde vocali ecc.

Un agente robotico può invece disporre di telecamere e telemetri a infrarossi come sensori, e di diversi motori come attuatori.

Definizione formale di agente razionale

Per ogni possibile sequenza di percezioni, un agente razionale dovrebbe scegliere un'azione che massimizzi il valore atteso della sua misura di performance, date le informazioni fornite dalla sequenza e da ogni ulteriore conoscenza posseduta dall'agente.

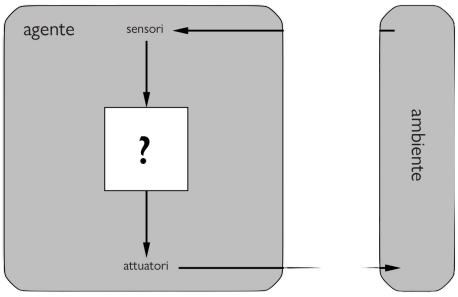

Figura 1: agente razionale

Useremo il termine **percezione** per indicare i dati che i sensori di un agente percepiscono.

La **seuquenza percettiva** di un agente è la storia completa di tutto ciò che esso ha percepito nella sua esistenza.

Sèecificando l'azione prescelta dell'agente per ogni possibile sequenza percettiva, abbiamo descritto l'agente in modo completo.

In termini matematici diciamo, quindi, che il comportamento di un agente è descritto dalla **funzione agente**, che descrive la corrispondenza tra una qualsiasi sequenza percettiva e una specifica azione:

$$f : \mathcal{P}^* \to \mathcal{A}$$

Possiamo immaginire di rappresentare in forma di tabella la funzione agente che descrive un certo agente.

Nella maggior parte dei casi questa tabella sarebbe molto grande, di fatto infinito, a meno di non specificare una lunghezza massima delle sequenza percettive che vogliamo prendere in considerazione.

La tabella è una descrizione esterna dell'agente.

Internamente, la funzione agente di un agente artificale sarà implementata da un **programma agente**.

E' importante tenere distinti questi due concetti:

- *funzione agente*: descrizione matematica stratta
- *programma agente*: implementazione concreta della funzione agente, in esecuzione all'interno di un sistema fisico.

Definizione

Se un agente ha $|P|$ possibili percezioni, quante voci avrà la funzione agente dopo T passi temporali?

$$\sum_{t=1}^{T} |P|^t$$

L'obiettivo dell'IA è di progettare piccoli programmi agente per rappresentare grandi funzioni agenti.

Qual'è il modo più corretto di progettare la tabella?, ovvero che cosa rende un agente buono o cattivo, intelligente o stupido?

Abbiamo bisogno di una **performance measure**.

Misure di prestazione

L'IA è fedele al concetto di **consequenzialismo**: vlautare il comportamento di un agente considernadone le *conseguenze*.

Quando un agente viene inserito in un ambiente, genera una sequenza di azioni in base alle percezioni che riceve.

Questa sequenza di azioni porta l'ambiente ad attraversare una sequenza di stati: se tale sequenza è *desiderabile*, significa che l'agente si è comportato bene.

Questa nozione di desiderabilità è catturata da una **misura di prestazione** (*performance measure*) che valuta una seuqenza di stati dell'ambiente.

Razionalità

Definizione: agente razionale

Per ogni possibile seuqenza di percezioni, un agente dovrebbe scegliere un'azione che massimizzi il valore atteso della sua misura di prestazione, date le informazioni fornite dalla sequenza percettiva e da ogni ulteriore conoscenza dell'agente.

Dobbiamo, però, distingure un paio di concetti:

razionalità $\neq$ onniscenza : le percezioni potrebbero non fornire tutte le informazioni rilevanti

razionalità $\neq$ chiaroveggente : i risultati dell'azione potrebbero non essere quelli previsti

razionalità $\neq$ riuscito

Razionale vuol dire *esplorazione*, *apprendimento*, *autonomia*.

Specificare un ambiente (PEAS)

Per progettare un agente intelligente bisogna definire l'ambiente in cui si opera:

- **performance measure**: come viene valutato il successo dell'agente
- **enviroment**: contesto in cui opera l'agente
- **actuators**: mezzi attraverso cui l'agente agisce sull'ambiente
- **sensors**: mezzi attraverso cui l'agente percepisce l'ambiente

Risolvere i problemi con la ricerca

Quando l'azione giusta da compiere non è subito evidente, un agente può avere la necessità di *guardare avanti*, cioè considerare una sequenza di azioni che formano un cammino che porterà a uno **stato obiettivo**.

Questo agente è chiamato **agente risolutore di problemi** e il processo computazionale che effettua è una **ricerca**.

Questo tipo di agente utilizza *rappresentazioni atomiche* in cui gli stati del mondo sono considerate come entità prive di una struttura interna visibile agli agli algoritmi per la risoluzione dei problemi.

Gli agenti che utilizzano *rappresentazioni di stati fattorizzate* o *strutture* sono chiamati **agenti pianificatori**.

Agenti risolutori di problemi

Gli **agenti risolutori di problemi** sono una forma ristretta di agenti che formulano un problema e un obiettivo partendo da uno stato, ignorando le percezioni.

Questo chiama *offline problem solving* perchè l'agente ha completa conoscenza dell'ambiente.

Problemi di ricerca e soluzini

...

Algoritmi di ricerca

Un **algoritmo di ricerca** riceve in input un problema e restituisce una soluzione o un fallimento.

Tree search

Consideriamo, quindi, algoritmi che sovrappongono un **albero di ricerca** al grafo dello spazio degli stati, formando vari cammini a partire dallo stato iniziale e cercando di trovarne uno che raggiunga uno stato obiettivo.

nodo : corrisponde a uno stato nello spazio degli stati

ramo : corrisponde ad un'azione

La radice dell'albero corrisponde allo stato iniziale del problema.

Nell'albero di ricerca possono esserci più cammini per raggiungere qualsiasi stato, ma per ogni nodo dell'albero c'è un cammino univoco per tornare alla radice.

Possiamo **espandere** il nodo, considerando le *azioni* disponibili per quello stato, usando la funzione *risultato* per vedere dove portano tali azioni e **generare** un nuovo nodo (*nodo figlio* o *sucessore*) per ognuno degli sttati risultanti.

Ogni nodo figlio ha un **nodo padre**.

Chiamiamo l'insieme di questi nodi **frontiera** dell'albero di ricerca, e diciamo che ogni stato per cui vi è un nodo generato è stato **raggiunto**.

La frontiera *separa* due regioni del grafo dello spazio degli stati:

- **regione interna**: ogni stato è stato espanso
- **regione esterna**: stati che non sono stati ancora raggiunti

Misurare le prestazioni nella risoluzione di problemi

Possiamo valutare le prestazioni di un algoritmo secondo quattro criteri:

- **completezza**: garantisce di trovare una soluzione (se esiste)
- **ottimalità**: garantisce di trovare la soluzione migliore (a costo minimo)
- **complessità temporale**: tempo che impiega a trovare una soluzione, calcolato in base al numero di nodi generati o espansi
- **complessità spaziale**: memoria necessaria per effettuare la ricerca, cioè il numero massimo di nodi memorizzati in memoria

Per essere completo, un algoritmo di ricerca deve procedere in modo **sistematico** nell'esplorazione di uno spazio degli stati infinito, assicurandosi di poter qualsiasi stato collegato a quello iniziale.

La complessità spaziale e quella temporale sono sempre considerate in rapporto a una misura della difficoltà del problema.

La misura tipica è la dimensione del grafo degli stati $= |V| + |E|$:

V : insieme dei vertici del grafo (stati)

E : insieme dei collegamenti o archi (coppie distinte stato/azione)

Tuttavia, in molti problemi di IA il grafo è rappresnetato soltanto *implicitamente* dallo stato iniziale, dalle azioni e dal modello di transizione.

Per uno spazio degli stati implicito, la complessità si può misurare in termini di:

- d (*depth*): profondità della soluzione meno costosa
- m: profondità massima dell'albero di ricerca
- b (*brach factor*): numero massimo di figli per nodo

Cammini rindondanti

Fallire nel riconoscere stati ripetuti può trasforare un problema lineare in un problema esponenziale.

Bisogna, quindi, mantenere una lista di stati già visitati e non espandere nodi che portano a stati già visitati.

Strategie di ricerca non informata

Un **algoritmo di ricerca non informata** non riceve alcuna informazione su quanto uno stato sia vicino all'obiettivo.

Es: consideriamo una persona che si trova ad Arad con l'obiettivo di raggiungere Burarest.

Una persona non informata, priva di conoscenza della geografia romena, non ha modo di sapre se sia meglio come primo passo andare a Zerind o Sibiu.

Ricerca in ampiezza

Quando tutte le azioni hanno lo stesso costo, una strategia appropriata è la **ricerca in ampiezza**, in cui si espande prima il nodo radice, poi tutti i suoi sucessori, poi i sucessori di quest'ultimi, e così via.

Potremmo implementare la ricerca in ampiezza, chiamata **RICERCA-BEST-FIRST**, con la funzione di valutezione $f(n)$ è uguale alla profondità del nodo, cioè al numero di azioni necessario per ragiungerlo.

Una coda FIFO (First In First Out) risulterà più veloce di una coda con priorità e ci fornirà l'ordine corretto dei nodi: i nuovi nodi vanno in fondo alla coda e quelli vecchi, a profondità minore, vengono espansi per primi.

Inoltre, raggiunti può essere un insieme di stati, anzichè una corrispondeza da stati a nodi, perchè una volta raggiunto uno stato, non possiamo più trovare un cammino migliore per raggiungerlo.

LA ricerca in amppiezza trova sempre una soluzione con un numero minimo di azioni, perchè quando genre nodi di profondità d, ha già generato tutti i nodi di profondità $d-1$, perciò se uno di essi fosse una soluzione, sarebbe stato stato trovato.

Ciò significa che è ottimale rispetto al costo per problemi in cui tutte le azioni hanno lo stesso costo, ma non per problemi che non hanno tale proprietà.

E' completa in ogni caso.

In termini di tempo di tempo e spazio, supponiamo di effettuare una ricerca in un albero uniforme dove ogni stato ha b sucessori.

Allora il numer totale di nodi generati è:

$$1 + b + b^2 + b^3 + \cdots + b^d = O(b^d)$$

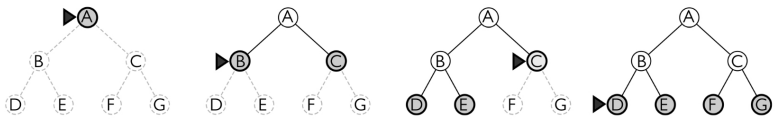

Figura 2: algoritmo BFS

```
function BFS(problem) returns a solution, or failure
  nodo <- nodo(problema.statoIniziale)
```

Tutti i nodi rimangono in memoria, perciò le complessità temporale e spaziale sono entrambe $O(b^d)$, quindi, esponenziali.

Nella ricerca in ampiezza, i requisiti di memoria rappresentano un problrma più importante rispetto al tempo di esucuzione.

Tuttavia, il tempo rimane comunque un fattore importante.

In generale, i problemi di ricerca con complessità esponenziale non possono essere risolti mediante ricercche non informate, se non nelle istante più pisccole.

Ricerca a costo minimo (algoritmo Dijkstra)

Quando le azioni hanno costi diversi, si utilizza BFS in cui la funzione di valutazione è il costo del cammino della radice al nodo corrente: si tratta dell'**algoritmo di Dijkstra** o **ricerca a costo uniforme**.

La complessità della ricerca a costo uniforme è caraterizzata in termini di C^* (costo della soluzione ottima) ed ϵ (limite inferiore imposto al costo di ogni azione), con $\epsilon > 0$.

caso peggiore (costi diversi): complessità temporale e spaziale $= O(b^{1+\lfloor C^*/\epsilon \rfloor})$

caso migliore (costi uniformi): complessità temporale e spazionale $= O(b^{d+1})$

La ricerca a costo uniforme è completa e ottima rispetto al costo perchè la prima soluzione che trova avrà un costo basso aleno quanto quello di ogni altro nodo sulla frontiera.

Questa riceerca esamina tutti i cammini in ordine di costo crescente, quindi, non rientra mai in un cammino infinito (assumendo che tutti i costi delle azioni $\epsilon > 0$).

Ricerca in profondità

La **ricerca in profondità** espande sempre per primo il nodo a *profondità maggiore* nella frontiera.

Potrebbe essere implementata con una chiamata di ricerca BFS in cui la funzione di valutazione f è l'opposto (negativo) della profondità.

Generalemente viene implementata come ricerca ad albero che non mantiene una tabella degli stati raggiunti.

La ricerca in profondità non è ottima rispetto al costo, infatti, restituisce sempre la prima soluzione che trova anche se non è la meno costosa.

Per spazi degli stati finiti che sono alberi, la ricerca in profondità è efficiente e completa; per spazi degli stati aciclici potrebbe arrivarre a espandere lo stesso stato più volte attraverso cammini diversi, ma (alla fine) esplorerà sistematicamente l'intero spazio.

In spazi degli stati ciclici, la ricerca in profondità può bloccarsi in un ciclo infinito, perciò alcune implementanzioni controllano ciascun nodo nuovo per verificare la presenza di cicli.

Infine, in spazi degli stati infiniti la ricerca in profondità non è sistematica: può entrare in un cammino infinito, anche se non vi sono cicli.

Quindi, la riceerca in profondità è incompleta.

Per i problemi in cui è praticabile un ricerca ad albero, la ricerca in profondità ha esigenze di memoria molto più ridotte.

Non viene mantenuta una tabella dei nodi raggiunti, e la frontiera è molto più piccola: mentre nella ricerca in ampiezza la frontiera può essere vista come la superficie di una sfera in continua espansione, nella ricerca in profondità la frontiera è solatanto un raggio della sfera.

Per uno spazio degli stati finito e a forma di albero una ricerca ad albero in profondità richiede un tempo proporzionale al numero degli stati e ha una complessità di memoria solo $O(bm)$, dove b è il fattore di ramificazione e m è la massima profondità dell'albero.

Ricerca a profondità limitata

Per evitare che la ricerca in profondità si èerda in un cammino infinito possiamo usare la **ricerca e profondità limitata**, in cui forniamo un limite di profondità, $\updownarrow$ e consideriamo tutti i nodi a profondità $\updownarrow$ come se non avessere sucessori.

La complessitàtemporale è $O(b^{\updownarrow})$ e quella spaziale è $O(b\updownarrow)$.

Se si sceglie male $\updownarrow$ l'algoritmo non riuscirà a trovare la soluzione, per cui è ancora incompleto.

Poichè la ricerca in profondità è una ricerca ad albero, non possiamo che perda tempo sui cammini ridondanti in generale, ma possiamo eliminare i cammini ciclici al posto di un pò di tempo di calcolo in più.

La **ricerca ad approfondimento iterativo** risolve il problema di trovare un buon valore di $\updownarrow$ provando tutti i valori: prima 0, poi 1, poi 2, e così via fino a quando si trova una soluzione, oppure la ricerca a profondità limitata restituisce il valore *fallimento* anzichè il valore *soglia*.

L'approfondimento iterativo riunisce molti vantaggi offerti dalla ricerca in profondità e da quella in ampiezza.

Come nella ricerca in profondità, i requisiti di memoria sono modesti: $O(bd)$ quando esiste una soluzione, $O(bm)$ su spazi degli stati finiti senza soluzione.

Come la ricerca in ampiezza, la ricerca ad approfondimento operativo è ottima per problemi in cui tutte le azioni hanno lo stesso costo, ed è completa su spazi degli stati aciclici

finiti, o su qualsiasi spazio degli stati finito quando controlliamo i nodi per la presenza di cicli risalendo l'intero cammino.

La complessità temporale è $O(b^d)$ quando esiste una soluzione, $O(b^m)$ quando non esiste.

Ogni iterazione della ricerca ad approfondimento iterativo genera un nuovo livello, come la ricerca in ampiezza, ma mentre quest'ultima lo fa memorizzando tutti i nodi in memoria, la ricerca ad approfondimento iterativo lo fa ricostruendo tutti i lielli precedenti, risparmiando memoria in cambio di un tempo maggiore.

In una ricerca ad approfondimento iterativo, i nodi al livello più basso (a profondità d) sono generati una sola volta, quelli al livello immediatamente superiore due volte, e così via fino ai figli diretti del nodo radice, che sono generati d volte.

Quindi, nel caso peggiore il numero totale dei nodi generati è:

$$N(RAI) = (d)b^1 + (d-1)b^2 + \cdots + (1)b^d$$

il che ci dà una complessità temporale $O(b^d)$, asintoticamente equivalente alla ricerca in ampiezza.

In gnerale, la ricerca ad approfondimento iterativo è il metodo preferito di ricerca non informata quando lo spazio degli stati in cui creare è troppo grande per essere mantenuto in memoria e la profondità della soluzione non è nota.

Criterio	**In ampiezza**	**A costo uniforme**	**In profondità**	**A profondità**
completa?	Si	Si	No	No
ottima rispetto al costo?	Si	Si	No	No
tempo	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^{\updownarrow}$
spazio	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(b^{\updownarrow}$

Per le ricerche sul garfo che effettuano tale controllo, le differnze principali sono che la ricerca in profodnità è completa per spazi degli stati finiti e che le complessità spaziale e temporale sono limitate dalla dimensione dello spazio degli stati (numero di vertici e archi, $|V| + |E|$).

Strategie di ricerca informata

La **ricerca informata** sfrutta conoscenza specifica del dominio applicativo per fornire suggerimenti su dove si potrebbe trovare l'obiettivo, possa trovare soluzini in modo più efficiente di una strategia non informata.

I sugggerimenti hanno la forma *funzione euristica* denotata con $h(n)$:[10]

$h(n)$ = costo stimato del cammino meno costoso dalla stato del nodo n a uno stato obiettivo

Ricerca best-first greedy o "golosa"

La **ricerca best-first grredy** è una forma di ricerca BFS che espande prima il nodo con il valore più basso di $h(n)$, cioè quello che appare più vicino all'obiettivo, sulla base del fatto che + probabile che questo porti rapidamente a una soluzione.

Perciò la funzione di valutazione è $f(n) = h(n)$.

La ricerca BFS greedy su grafo è completa negli spazi degli stati fniti, ma non in quelli infiniti.

Nel caso peggiore la complessità temporale e spaziale è $O(|V|)$, ma con una buona funzione curistca è possibile ridurla considervolmente, arrivando in certi problemi a $O(bm)$.

Ricerca A*

La forma più diffusa di algoritmo di ricerca informata è la **ricerca A*** una BFS che utilizza la funzione di valutazione:

$$f(n) = g(n) + h(n)$$

$g(n)$: costo del cammino dal nodo iniziale al nodo n

$h(n)$: costo stimato dal cammino più breve da n a uno stato obiettivo

$f(n)$: costo stimato del cammino migliore che continua da n fino a un obiettivo

La ricerca A* è completa.

Il fatto cche sia o meno ottimale rispetto a costo dipende da alcune proprietà dell'euristica.

Una proprietà fondamentale è l'**ammissibilità**: un'**euristica ammissibile** è tale se non sovrastima mai il costo per raggiungere un obiettivo.

Con un'euristica ammissibile la riceerca A* è ottima rispetto al costo, cosa che possiamo dimostrare per assurdo.

Un'euristica $h(n)$ si dice **consistente** (o monotona) se, per ogni nodo n e ogni sucessore n' generato da n mediante un'azione a, il costo stimato di raggiungere l'obiettivo da n è minore o uguale al costo per raggiungere n' più il costo stimato di raggiungere l'obiettivo da n':

$$h(n) \leq c(n, a, n') + h(n')$$

Questa è una forma della **disugualianza triangolare**, per cui ogni lato dei un triangolo deve essere minore o uguale alla somma degli altri due lati.

Ogni euristica consistente è anche ammissibile (non viceversa), perciò la ricerca A* con un'euristica consistente è ottima rispetto al costo.

Con un'euristica consistente, la prima volta che raggiungiamo uno stato sarà su un cammino ottimo, perciò non dovremo mai aggiungere di nuovo uno stato alla frontiera, nè dovremo mai modificare un elemento in raggiunti.

Funzioni euristiche

Effetto dell'accuratezza dell'euristica sulle prestazioni

I risultati mostrano che h_2 è più accurata di h_1, e che entrambe sono miglirori di non usare alcuna euristica ($h(n) = 0$).

Ci si potrebbe chiedere se h_2 sia sempre migliore di h_1 e la risposta è *si*: dalle definizioni di h_1 e h_2 risulta che per ogni stato n (entrambi ammissibili), $h_2(n) \geq h_1(n)$.

diciamo, quindi che h_2 **domina** h_1.

La dominazione si traduce dirrettamente in effecienza: la ricerca A* che usa h_2 non espanderà mai più nodi di quella che usa h_1.

Se ne deduce che è generalmente meglio usare una funzione euristica con valori più alti, a patto che sia consistentee che il tempo per calcolarla non sia troppo lungo.

Generare euristicheda problemi rilassati

Com'è possibile invenatre un'euristica come h_2?

h_1 e h_2 sono stime della lunghezza del cammino minimo da uno stato a quello obiettivo, ma sonno anche lunghezze di cammino perfettamente accurate per versioni semplificate del rompicapo.

Se le regole del problema fossero cambiate in modo che un tassello potesse muoversi ovunque (invece che solo nello spazio vuoto), il costo minimo per raggiungere l'obiettivo sarebbe esattamente h_1.

In modo analogo, se un tassello potesse muoversi di uno spazio in ogni direzione, anche se la posizione fosse già occupata da un altro tassello, il costo minimo per raggiungere l'obiettivo sarebbe esattamente h_2.

Un problema con meno vincoli sulle azioni è detto un **problema rilassato**.

Il grafo dello spazio degli stati per il problema rilassato è un supergrafo di quello dello spazio degli stati originale, perchè la rimozione dei voncoli crea nuovi archi nel grafo.